

Computing Platform for Autonomous Driving (CP4AD)

2. RTOS #1

Prof. Jong-Chan Kim

Dept. Automobile and IT Convergence



국민대학교
KOOKMIN UNIVERSITY

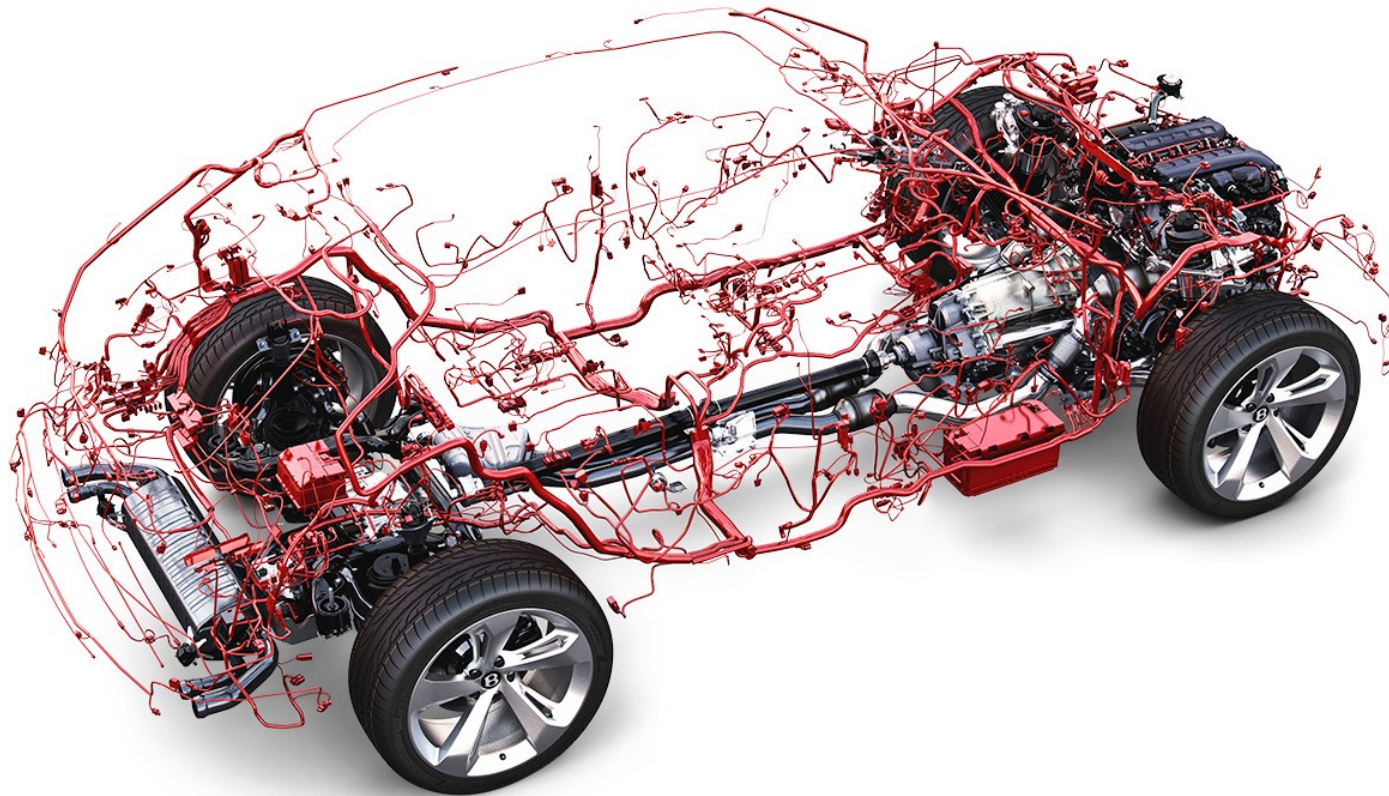
Real-Time Embedded System

Real-Time Embedded Systems

- Real-Time System
 - A computing system that cares both logical and temporal correctness
 - Logical correctness: produces correct outputs
 - Temporal correctness: produces outputs at a right time
 - Usually a “embedded system”
- Embedded System
 - A computing system that is a part of a larger system with a dedicated function
 - Usually hidden from users
 - Usually a “real-time system”

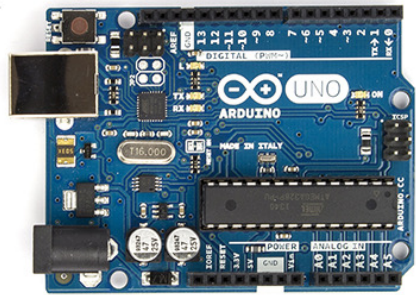
Automotive System

- A complex **Distributed Hard Real-Time Embedded System**
 - Dozens of ECUs (Electronic Control Units) connected through networks
- Temporal correctness is defined as end-to-end delays
 - From sensors to actuators through ECUs and networks



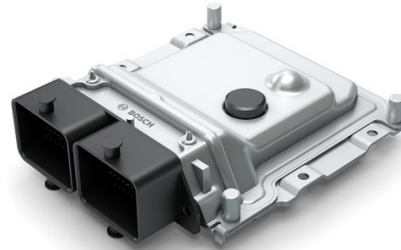
Various Automotive OSes

Firmware



No OS

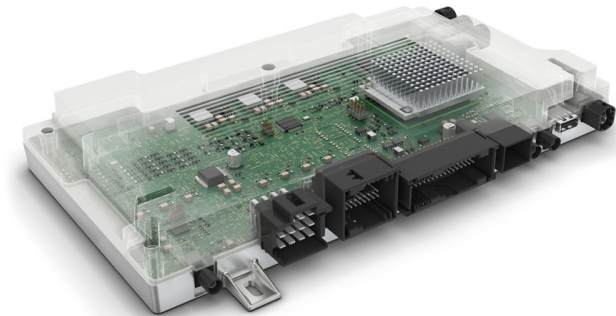
Real-Time OSes



AUTOSAR

Classic Platform

POSIX OSes (Linux, QNX, ...)



AUTOSAR

Adaptive Platform

Firmware vs. RTOS

Firmware (Superloop)

```
main()
{
    system_init();
    while(1) {
        do_A();
        do_B();
        do_C();
        ...
    }
}
```

RTOS-based

```
TASK(5ms)
{
    do_A();
    TerminateTask();
}

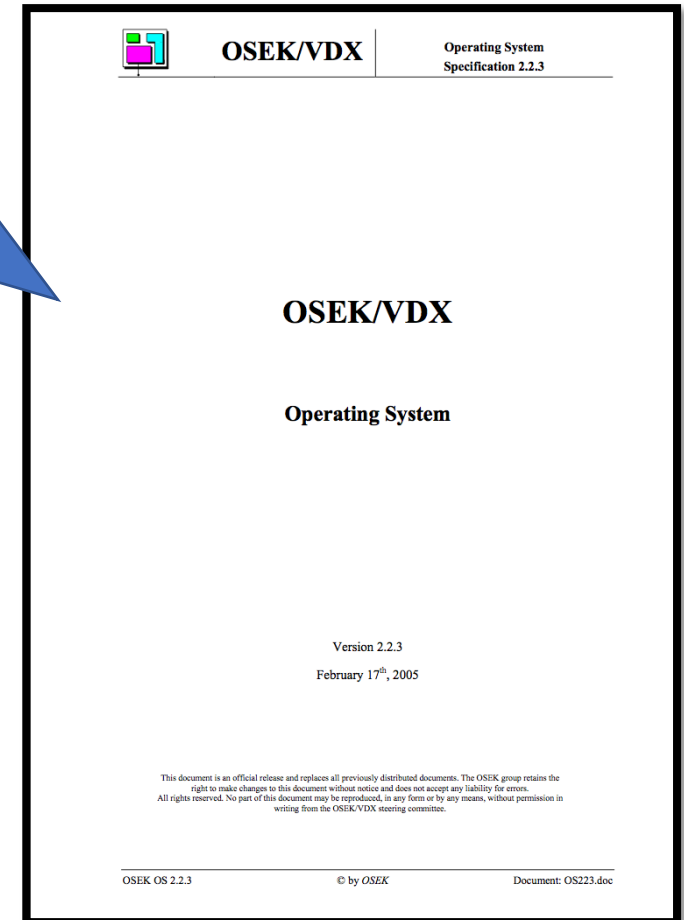
TASK(10ms)
{
    do_B();
    TerminateTask();
}

TASK(50ms)
{
    do_C();
    TerminateTask();
}
```

OSEK/VDX Specification

- OSEK
 - Offene Systeme und deren Schnittstellen für die Elektronik in Kraftfahrzeugen
 - “Open Systems and their Interfaces for the Electronics in Motor Vehicles”
 - German standard
- VDX
 - Vehicle Distributed eXecutive
 - French standard
- OSEK/VDX
 - A merge of OSEK and VDX
 - Industry standard RTOS spec. for automotive systems

A spec, not an implementation



AUTOSAR OS

- An extension of OSEK/VDX in the AUTOSAR standard
- Extended features
 - Schedule table
 - Timing protection
 - Memory protection

Document Title	Specification of Operating System
Document Owner	AUTOSAR
Document Responsibility	AUTOSAR
Document Identification No	034
Document Classification	Standard
Document Status	Final
Part of AUTOSAR Release	4.2.2

Document Change History		
Release	Changed by	Change Description
4.2.2	AUTOSAR Release Management	• Allow calls to ControlIdle from all cores • Minor updates/clarification of descriptions • Editorial changes
4.2.1	AUTOSAR Release Management	• Add support for AsilQmProtection • Minor updates/clarification of descriptions • Editorial changes
4.1.3	AUTOSAR Release Management	• Changed multiplicity of attributes in locSender/ReceiverProperties • Minor updates/clarification of descriptions • Editorial changes
4.1.2	AUTOSAR Release Management	• Clarification on E_OS_NESTING_DEADLOCK • Update of table 2 • Corrected multiplicity of ECUC_Os_00393 • Minor updates/clarification of descriptions • Editorial changes • Removed chapter(s) on change documentation
4.1.1	AUTOSAR Administration	• Add support for ECU degradation • Changed service interface description to a formal format • Several minor changes and clarifications
4.0.3	AUTOSAR Administration	• Included MultiCore support from former "Specification of Multi-Core OS Architecture"
3.1.5	AUTOSAR Administration	• Clarification in 7.8.1 (meaning of "do nothing") and 7.1.2.1 ("OSEK declarations") • Minor changes as typos and rewording
3.1.4	AUTOSAR Administration	• Extension of services (Chapter 12) • States in OS- Applications • Active termination of other OS-Applications in possible (Chapter8) • Legal disclaimer revised • Chapter 10.4 revised

Real-Time Scheduling Theory

- The whole story begins from a single paper in 1973
 - Periodic task model
 - L&L (Liu and Layland) bound
- Theoretical foundations behind
 - Real-time operating system (RTOS)
 - AUTOSAR OS
- We will learn
 - Periodic task model
 - Various scheduling algorithms
 - Various schedulability analysis methods

Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment

C. L. LIU

Project MAC, Massachusetts Institute of Technology

AND

JAMES W. LAYLAND

Jet Propulsion Laboratory, California Institute of Technology

ABSTRACT. The problem of multiprogram scheduling on a single processor is studied from the viewpoint of the characteristics peculiar to the program functions that need guaranteed service. It is shown that an optimum fixed priority scheduler possesses an upper bound to processor utilization which may be as low as 70 percent for large task sets. It is also shown that full processor utilization can be achieved by dynamically assigning priorities on the basis of their current deadlines. A combination of these two scheduling techniques is also discussed.

KEY WORDS AND PHRASES: real-time multiprogramming, scheduling, multiprogram scheduling, dynamic scheduling, priority assignment, processor utilization, deadline driven scheduling

CR CATEGORIES: 3.80, 3.82, 3.83, 4.32

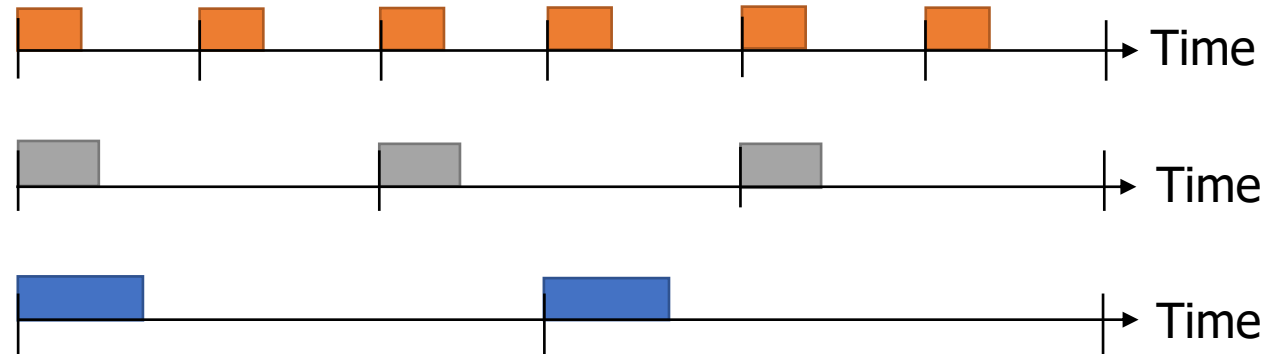
Scheduling algorithms for multiprogramming in a hard-real-time environment

CL Liu, JW Layland - *Journal of the ACM (JACM)*, 1973 - dl.acm.org

... **scheduling algorithm** is a set of rules that determine the task to be executed at a particular moment. The **scheduling algorithms** ... Thus the specification of such **algorithms** amounts to the ...

★ Save Cite Cited by 13592 Related articles All 77 versions

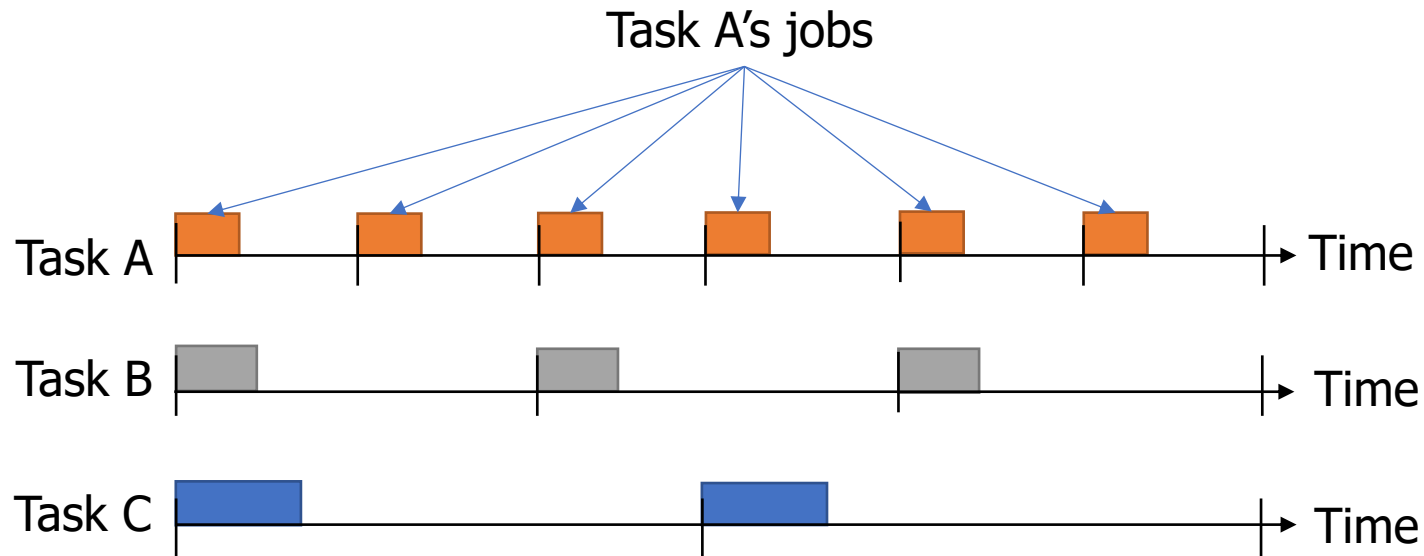
Workload Model: Key Concepts



- Task and jobs
- Task offset
- Release time of jobs (periodic, sporadic, aperiodic)
- Deadline of jobs (hard or soft)
- Execution time of jobs (deterministic or stochastic)

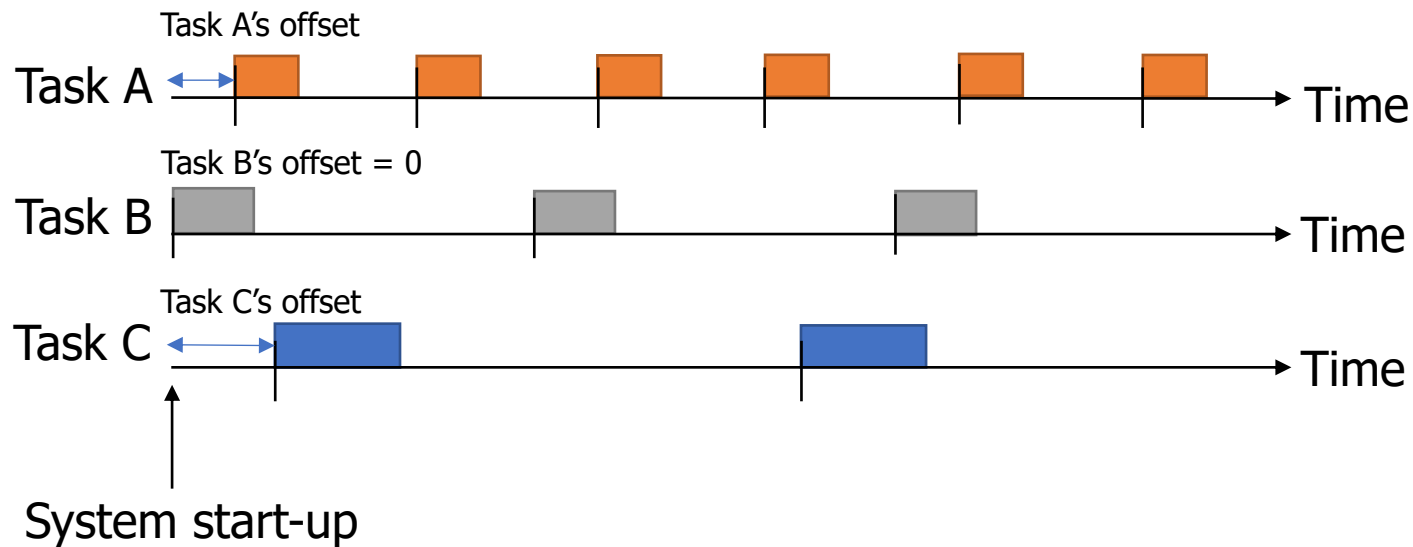
Jobs and Tasks

- A **job** is a unit of work, e.g.,
 - Displaying the current RPM
 - Reading the current speed sensor data
- A **task** is a sequence of the same jobs, e.g.,
 - RPM displaying task
 - Speed sensor data reading task



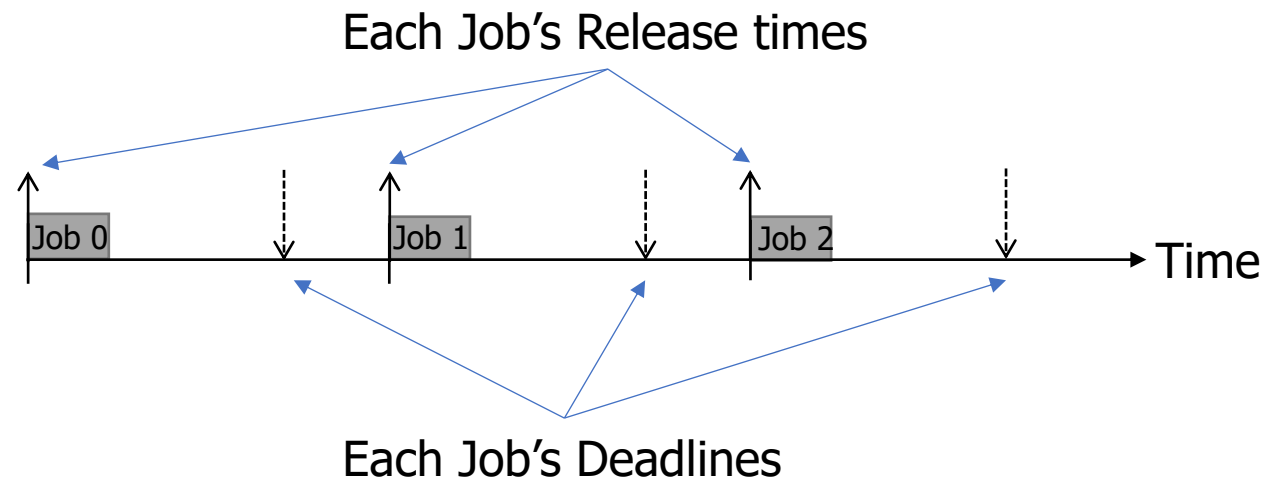
Task Offset

- Time between the system initialization and the first job release
- Tasks in a system may have different offsets



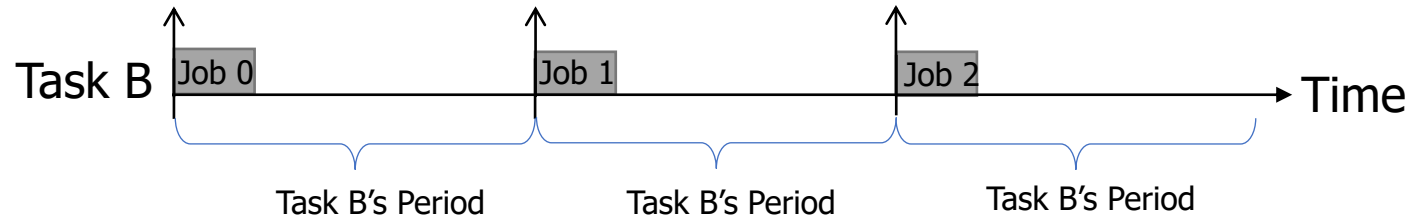
Release Time and Deadline

- Release time of a job
 - The time instant at which the job becomes ready to run
- Deadline of a job
 - The time instant by which the job should complete

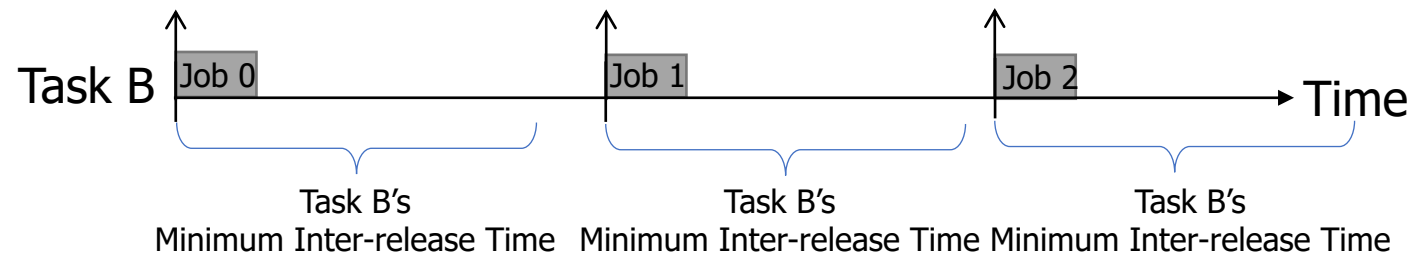


More about release times (1/2)

- A task is **periodic**, if the releases are strictly periodic, e.g., every 100ms

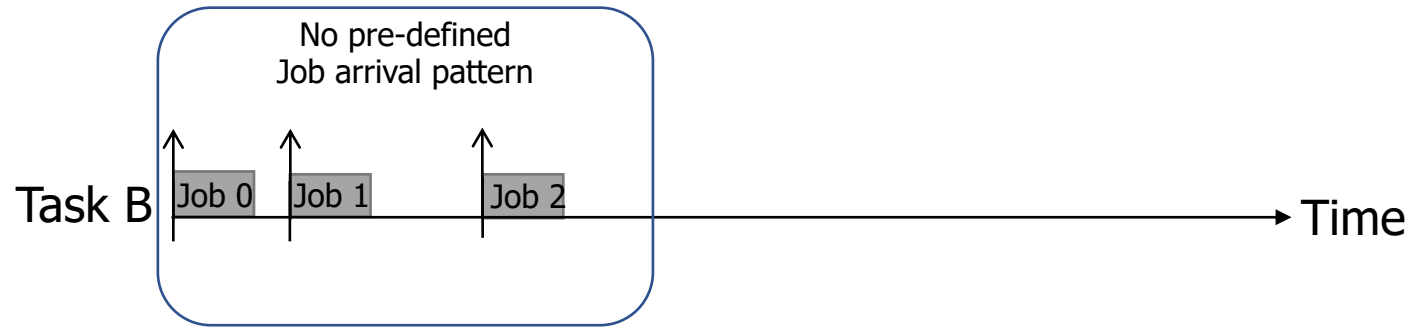


- A task is **sporadic**, if its jobs are not periodic but released with a minimum inter-release time



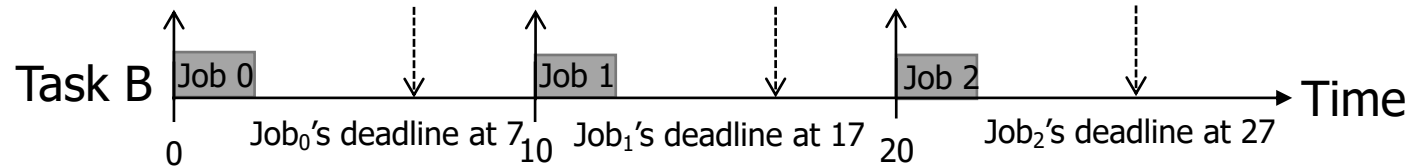
More about release times (2/2)

- If jobs arrive irregularly, it is said to be **aperiodic**

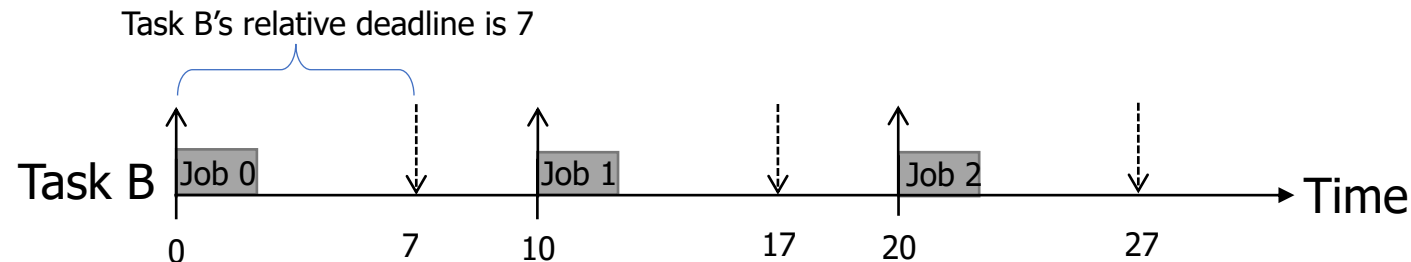


More about deadlines (1/3)

- Absolute deadline
 - When the deadline is specified in absolute time

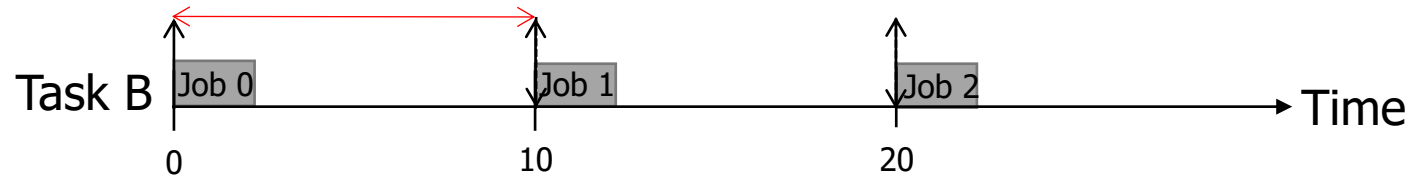


- Relative deadline
 - Expressed as a time distance from the job release time

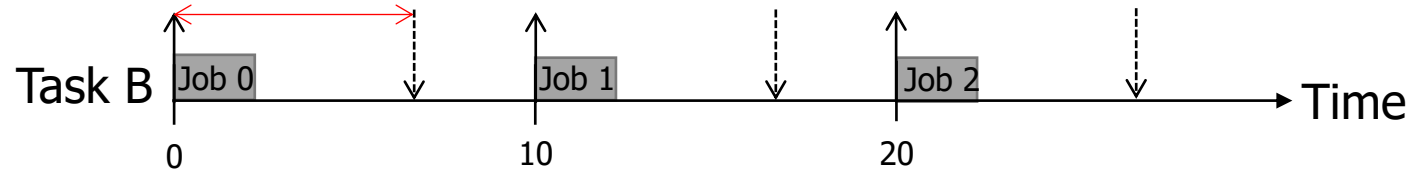


More about deadlines (2/3)

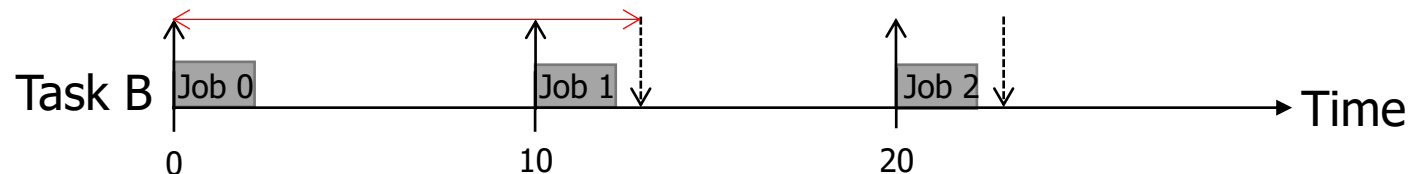
- Implicit deadline
 - Deadline = period



- Constrained deadline
 - Deadline $\leq$ period



- Arbitrary deadline
 - Deadline can be larger than the period

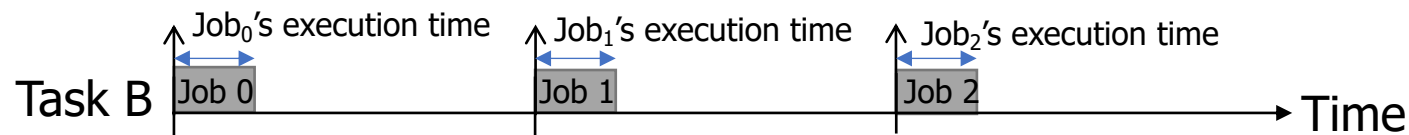


More about deadlines (3/3)

- Hard deadlines
 - If failing to meet them is considered as an application failure leading to catastrophic consequences on the controlled environment, it is called “hard deadline”
- Soft deadlines
 - When late completions degrade performance without causing serious damages, it is called “soft deadline”
 - Usually specified in probability. For example, human eyes cannot notice when the media player’s deadline miss ratio $< 10\%$

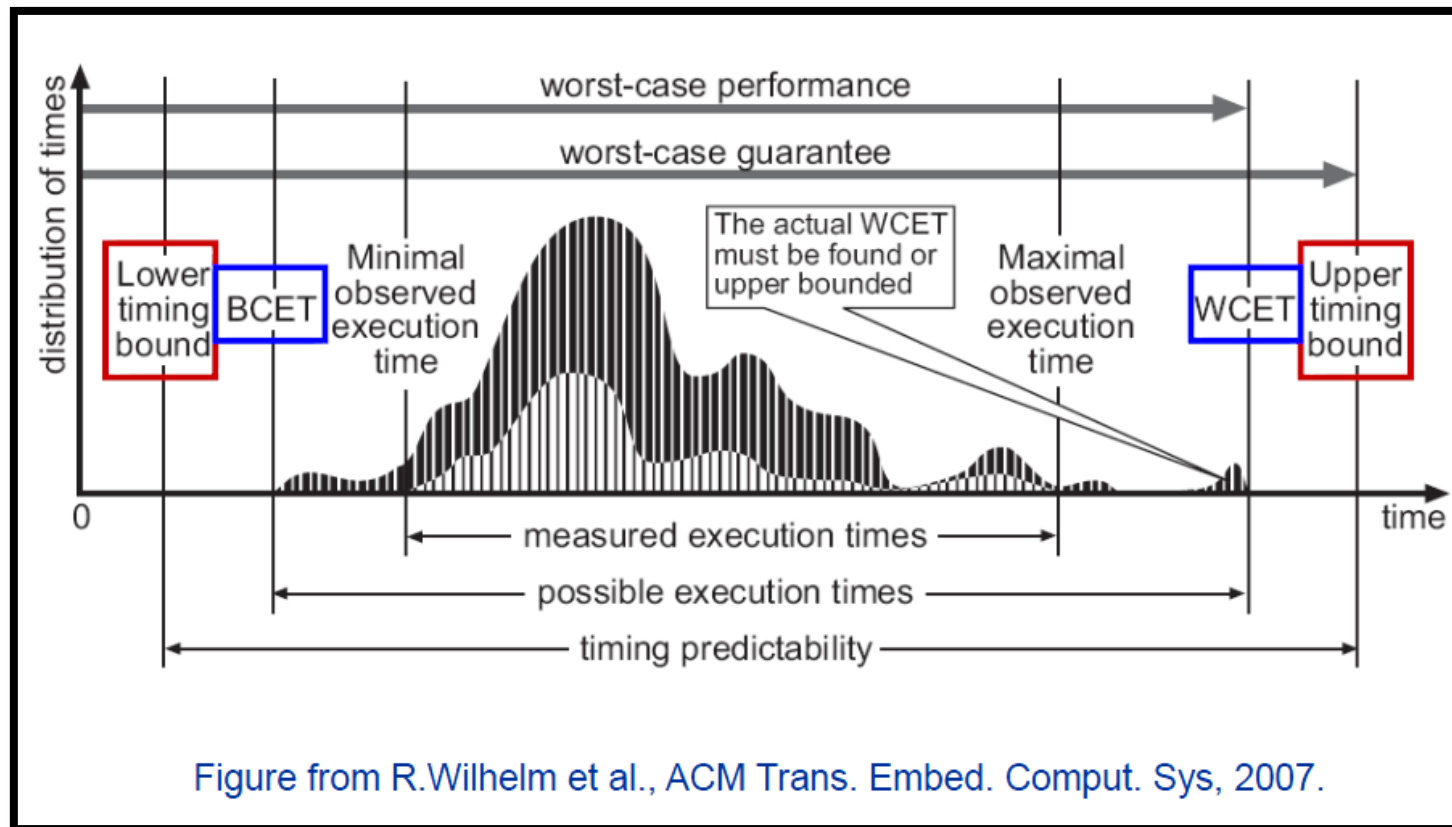
Execution Times

- Each job's execution time
 - Same jobs may have different execution times due to different inputs and system states
- Each task's execution time
 - Then, how to model each task's execution time?



More on execution times

- Stochastic execution time
 - Task execution time can be expressed as a probabilistic distribution
- Deterministic execution time
 - Usually, only the worst-case execution time (WCET) is meaningful

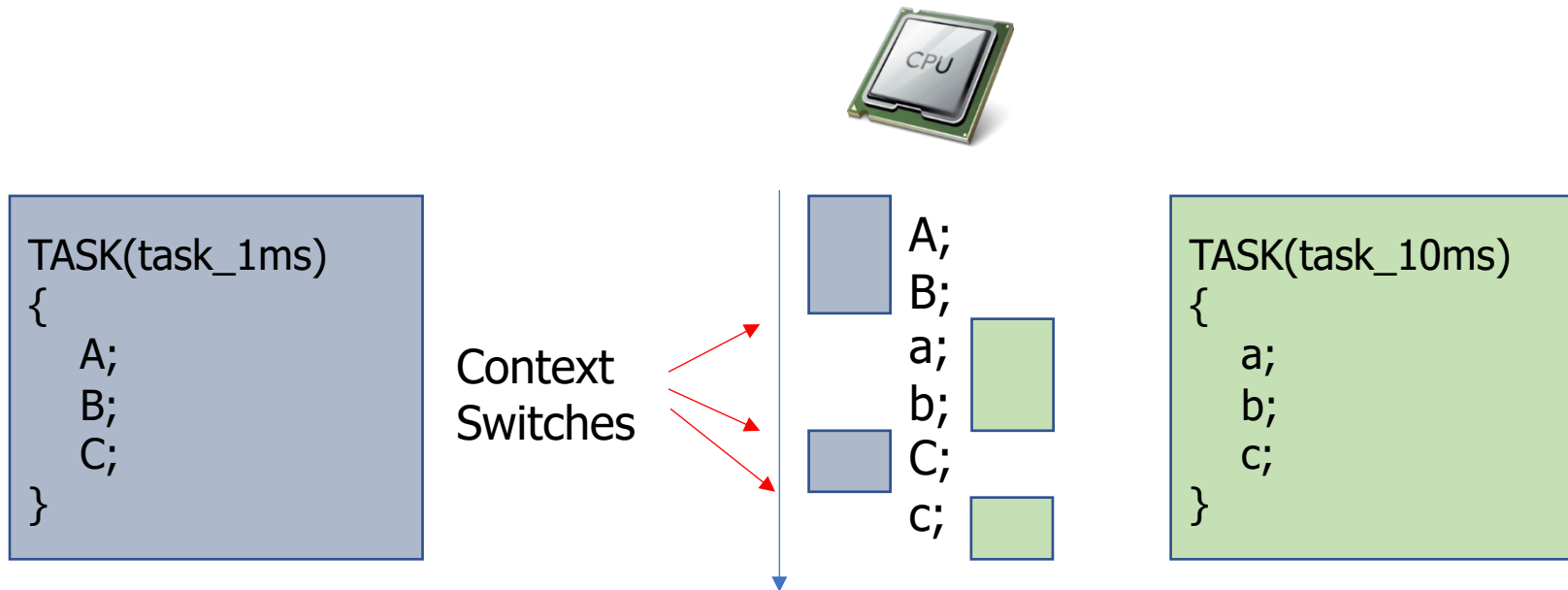


Our Assumed Workload: Periodic Task Model

- We have a set of N periodic tasks
 - $\{T_1, T_2, \dots, T_N\}$
- Each T_i is generally represented by a three-tuple (p_i, e_i, d_i)
 - p_i : period
 - e_i : WCET (Worst-Case Execution Time)
 - d_i : **hard** relative deadline
- When T_i is an implicit-deadline task
 - (p_i, e_i)

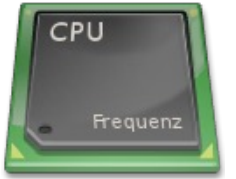
Resource Model: A Singlecore CPU

- For now, we assume a singlecore CPU
 - Shared by multiple periodic tasks
 - Preemptable
 - Most CPU provides context restore/save mechanisms to support preemptive scheduling

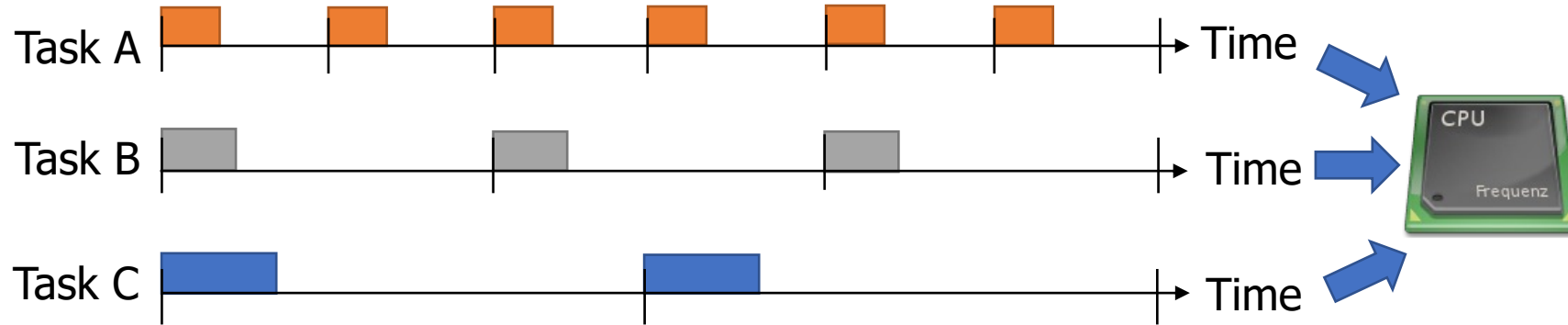


Preemptable vs. Non-Preemptable Resources

- Preemptable resource
 - Even when a certain job is already executing on a resource, it can be suspended at arbitrary points to start another job
 - The suspended job can be resumed without a significant penalty
- Non-preemptable Resource
 - Once a job or message starts executing or being transferred on resource, it cannot be suspended until it voluntarily releases the resource

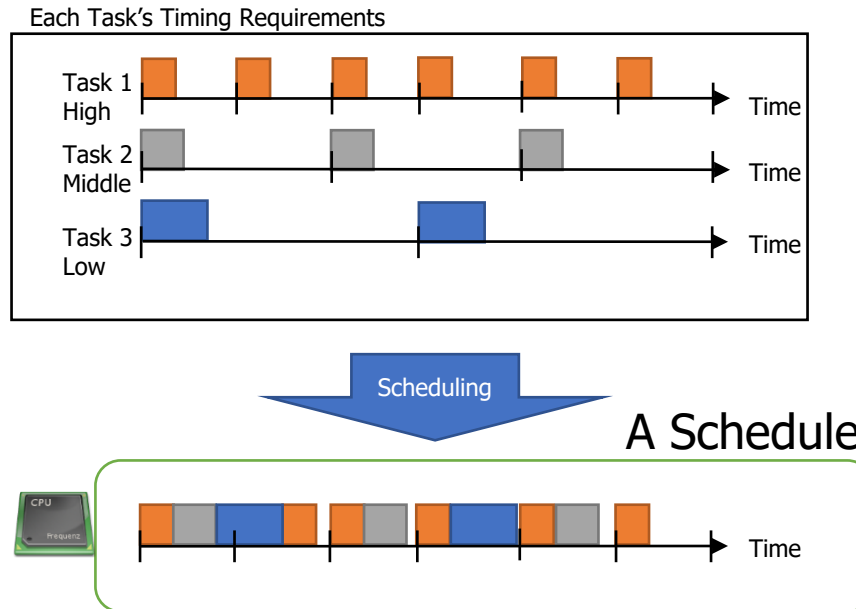


Handling Multiple Tasks with a Single Resource



Scheduling Algorithm

- Schedule
 - Assignment of jobs to the resource in the time domain



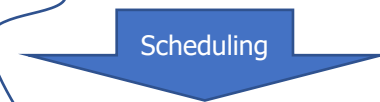
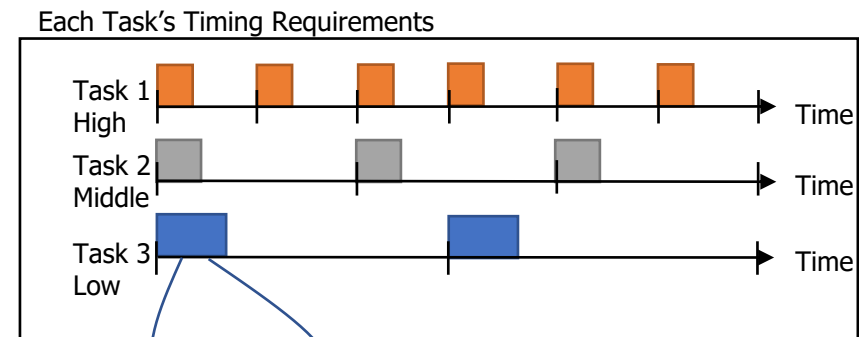
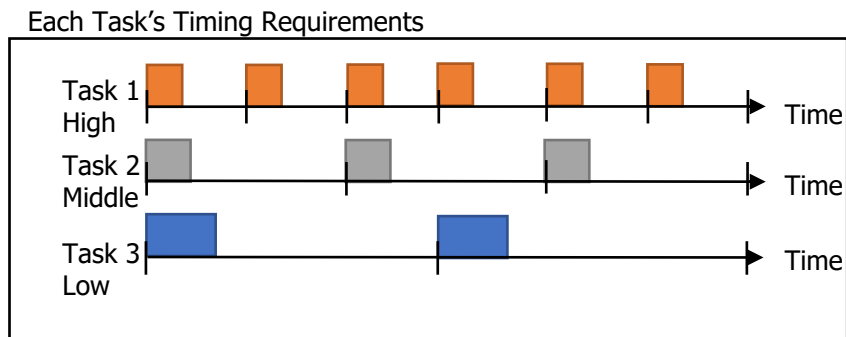
- Scheduler
 - A module that implements the scheduling algorithm

Scheduling for what?

- Non-real-time scheduling by GPOS
 - Maximize average **performance**
 - Minimize response time
 - Maximize throughput
 - Ensure fairness
 - No starvation
- Real-time scheduling
 - Ensure **predictability**
 - Meet every job's deadline (HARD)
 - Maintain deadline meet ratio under a threshold (SOFT)

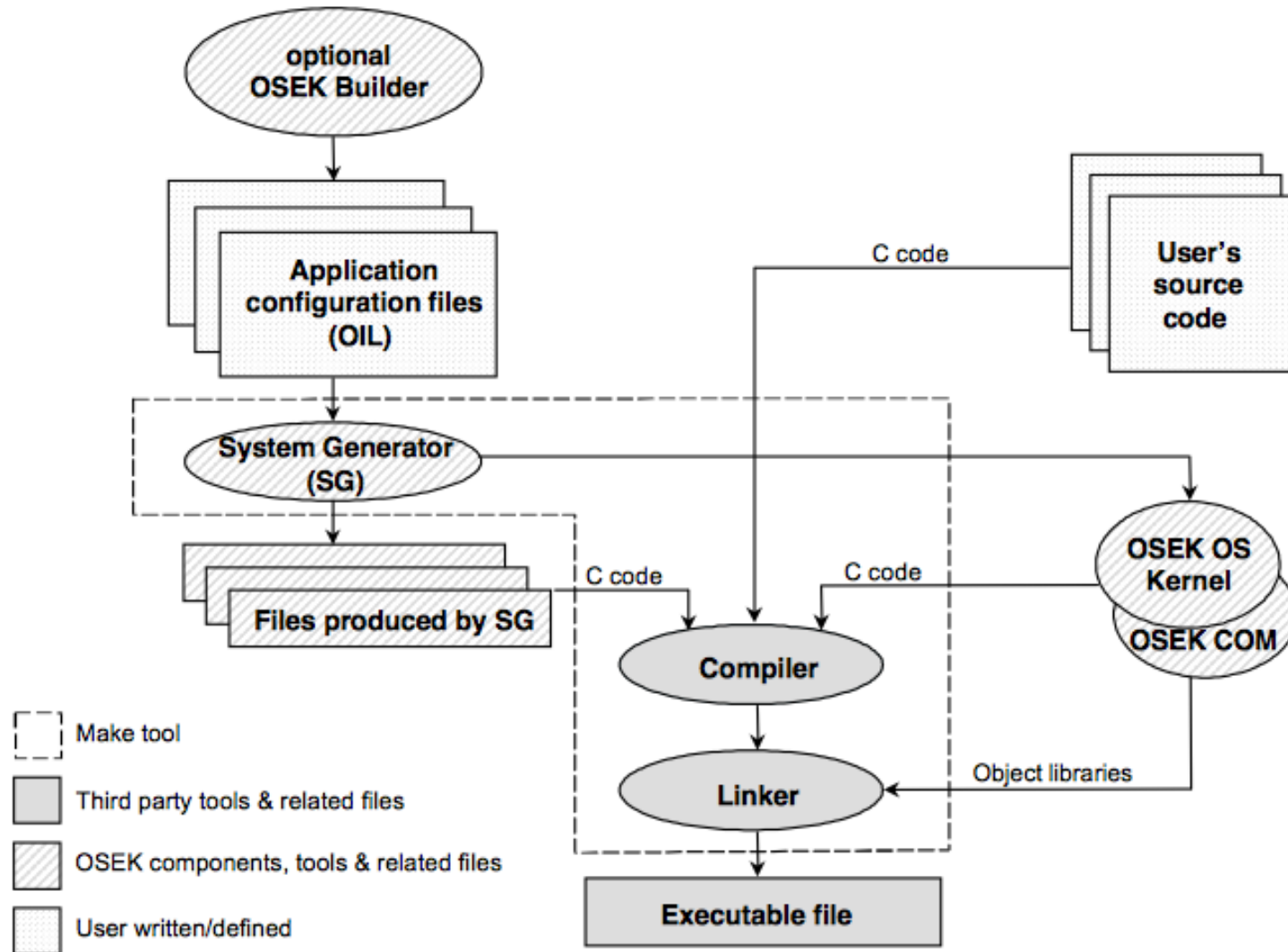
Preemptive vs. Non-Preemptive Scheduling

- Preemptive Scheduling
 - You can arbitrarily suspend the currently executing task
- Non-preemptive Scheduling
 - You have to wait for the current task to finish before starting a new task
- Cooperative Scheduling
 - Preemption allowed only at pre-defined time points



OSEK/VDX Real-Time Operating System

OSEK/VDX Build Process



Erika Enterprise Build Process

```
Jong-Chanui-iMac:Debug jongchank$ make
Using erika files in /Users/jongchank/EE_RT-Druid-2.7.0-juno-macosx-cocoa-x86_64
/eclipse/plugins/com.eu.evidence.ee.2.7.0.20160615_1643/ee_files
MAKE_DIRECTORIES
CC hooks.c
CC wiring.c
CC wiring_digital.c
CC wiring_analog.c
CC wiring_pulse.c
CC wiring_pulse.S
CC wiring_shift.c
CC WInterrupts.c
CCXX abi.cpp
CCXX CDC.cpp
CCXX HardwareSerial.cpp
CCXX HardwareSerial0.cpp
CCXX HardwareSerial1.cpp
CCXX HardwareSerial2.cpp
CCXX HardwareSerial3.cpp
CCXX IPAddress.cpp
CCXX new.cpp
CCXX PluggableUSB.cpp
CCXX Print.cpp
CCXX Stream.cpp
CCXX Tone.cpp
CCXX USBCore.cpp
CCXX WMath.cpp
CCXX WString.cpp
CCXX Timer1.cpp
AR libarduiniosdk.a
CC eecfg.c
CCXX code.cpp
CCXX serial.cpp
CC task.c
CC ee_avr8_irstub.c
CC ee_avr8_utils.c
CC ee_avr8_oo.S
CC ee_hal_structs.c
CC ee_context.c
CC ee_activate.c
CC ee_schedule.c
CC ee_force_schedule.c
CC ee_gettaskstate.c
CC ee_gettaskid.c
CC ee_terminate.c
CC ee_thendin.c
CC ee_irstendin.c
CC ee_iparam.c
CC ee_tstub.c
CC ee_tnterm.c
CC ee_lookup.c
CC ee_rq_exchg.c
CC ee_rq_inser.c
CC ee_shtdown.c
CC ee_startos.c
CC ee_rq_first.c
CC ee_chaintas.c
CC ee_lockres.c
CC ee_unlockres.c
CC ee_ulockallres.c
CC ee_alcancel.c
CC ee_algetbase.c
CC ee_alget.c
CC ee_alsetabs.c
CC ee_alsetrel.c
CC ee_altick.c
CC ee_getcountervalue.c
CC ee_getelapsdvalue.c
CC ee_intvect.c
AR libee.a
LD arduino.elf
text data bss dec hex filename
5848 464 208 6520 1978 arduino.elf
EEP arduino.elf
HEX arduino.elf
LSS arduino.elf
SREC arduino.elf
*****
Compilation terminated successfully!
Jong-Chanui-iMac:Debug jongchank$
```

Arduino HW Drivers

Generated from oil

User Sources

Erika Enterprise Kernel Sources

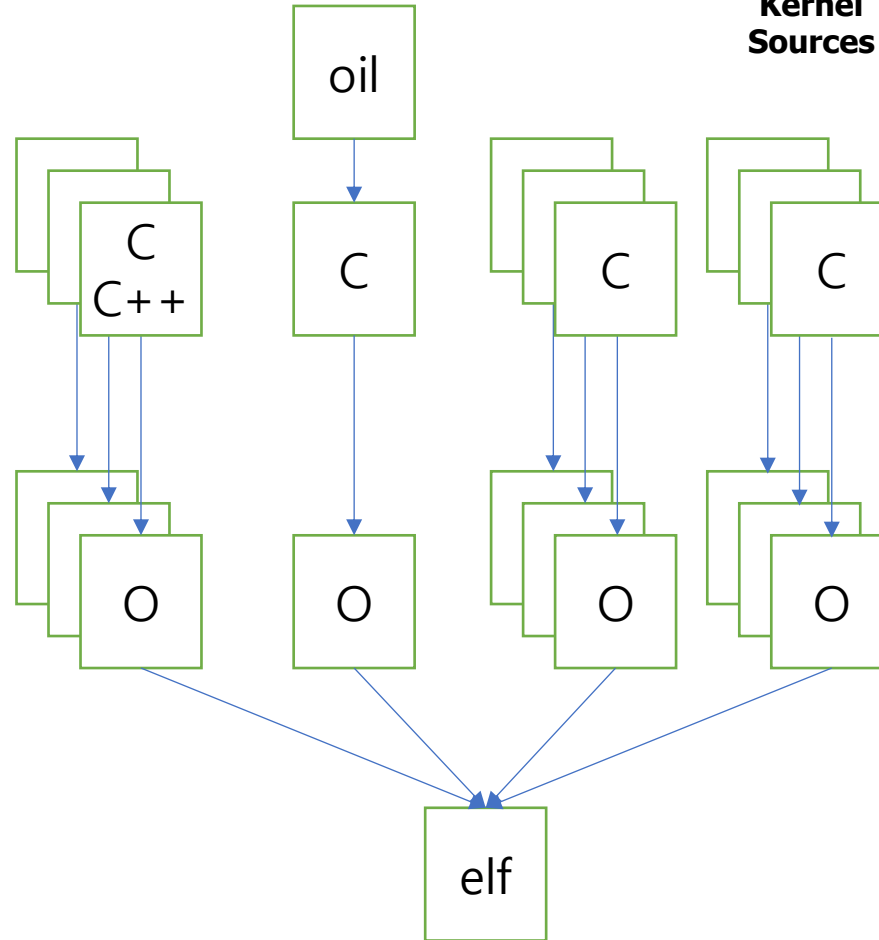
Final Executable

Arduino
HW Drivers

Generated
from oil

User
Sources

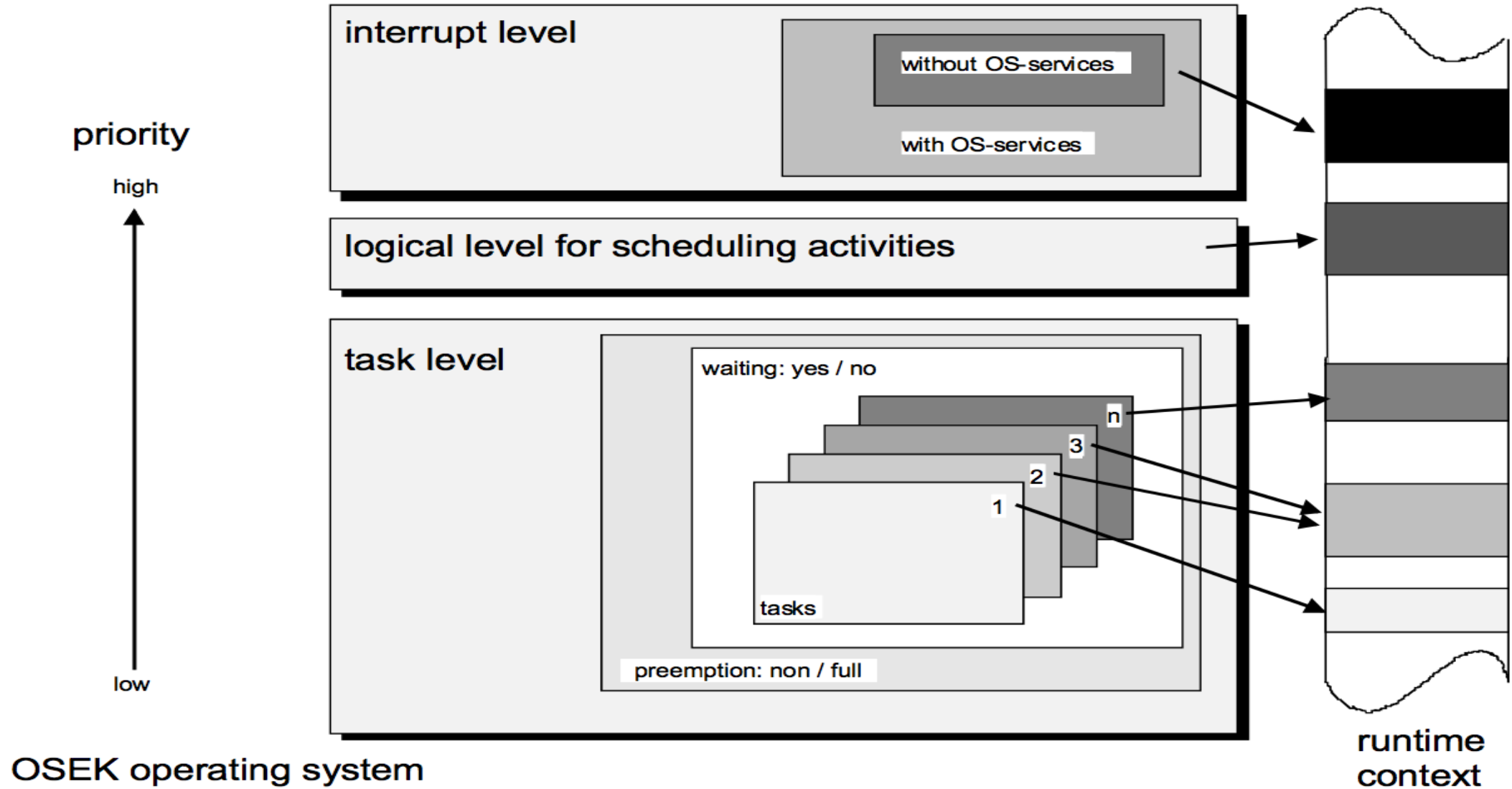
Erika
Enterprise
Kernel
Sources



OSEK OS Objects Overview

- Task
 - Unit of function or piece of code
 - OS Controlled
- ISR (Interrupt Service Routine)
 - Interrupt Handler
 - HW Triggered
- Resources
 - Lock which makes critical section
- Counter
 - Abstraction of HW timer
- Alarms
 - Enables time-aware applications
- Events
 - Used for task synchronization

Processing Levels



Making a Task

- C code

```
TASK(TaskL1) {  
    ...  
    TerminateTask();  
};
```

Task ID

Task Body

End of Task

- OIL code

```
TASK TaskL1 {  
    PRIORITY = 1;  
    STACK = SHARED;  
    SCHEDULE = FULL;  
    AUTOSTART = FALSE;  
    ACTIVATION = 1;  
};
```

Task ID

Task Priority (Larger is Higher)

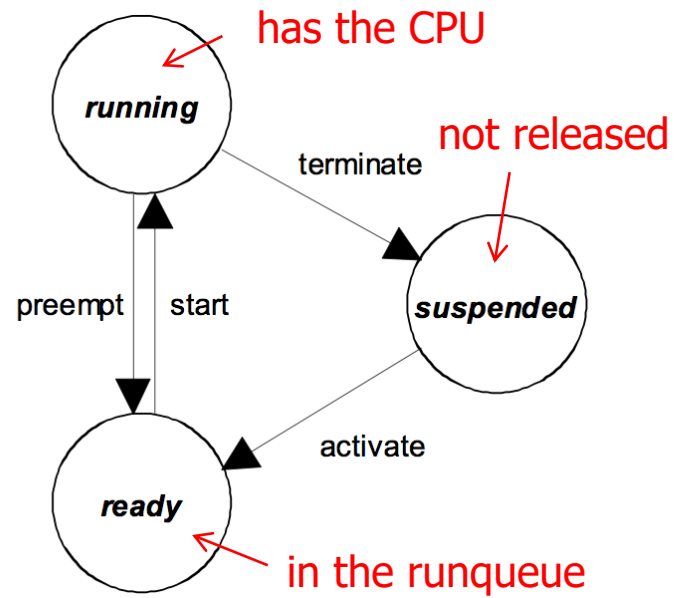
Use Shared Stack

Automatic Start

- FULL : preemptable
- NON: non-preemptable

Maximum number of queued activations

Task States



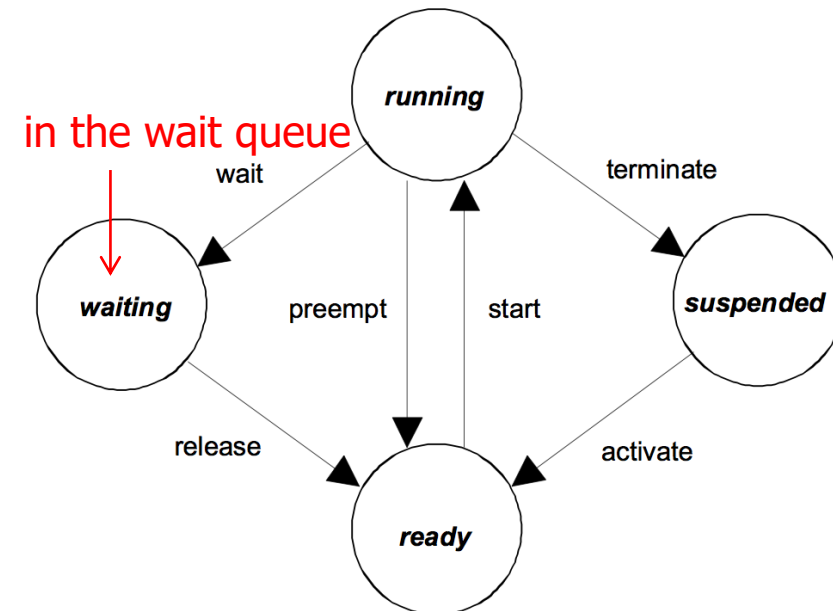
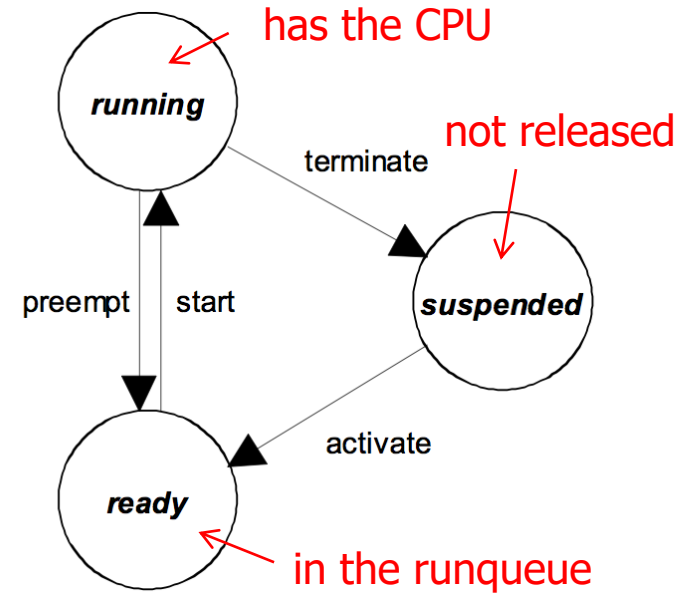
Two Task Types

- Basic Task
 - Waiting state not allowed

```
TASK(myTask)
{
    ...
    TerminateTask();
}
```

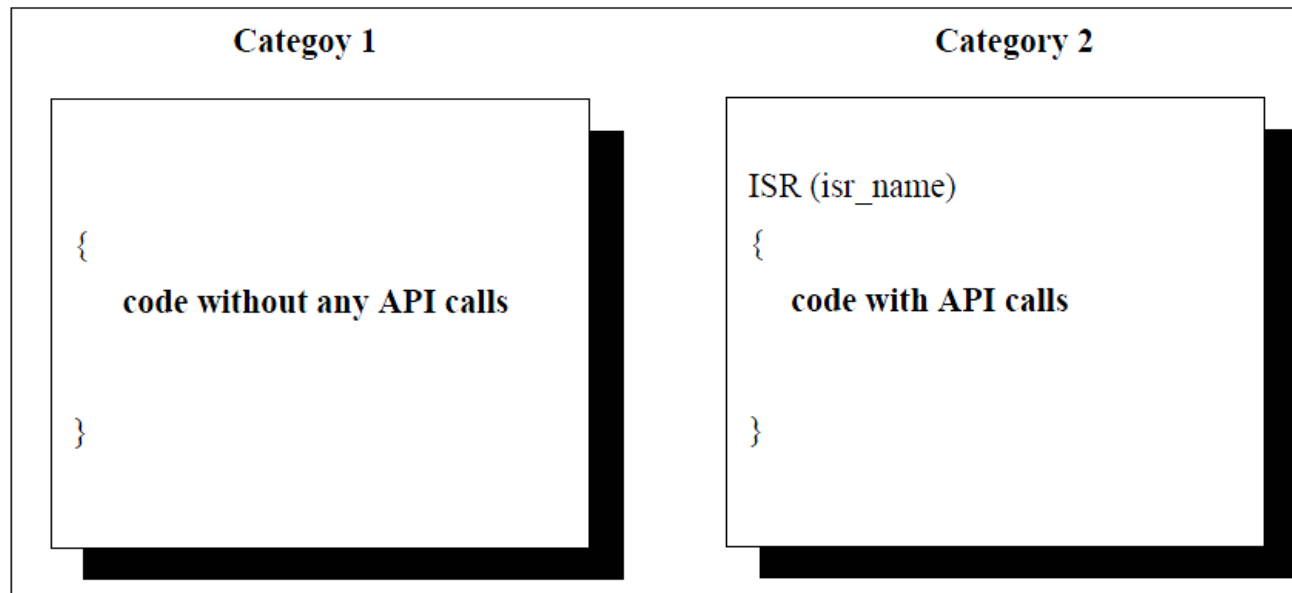
- Extended Task
 - Waiting state allowed

```
TASK(myTask)
{
    ...
    WaitEvent(event);
    ...
    TerminateTask();
}
```



Interrupt Handling

- ISR (Interrupt Service Routine)
 - A function which is triggered by an interrupt
- Two ISR categories
 - ISR category 1: do not use any OS service
 - ISR category 2: may call OS services

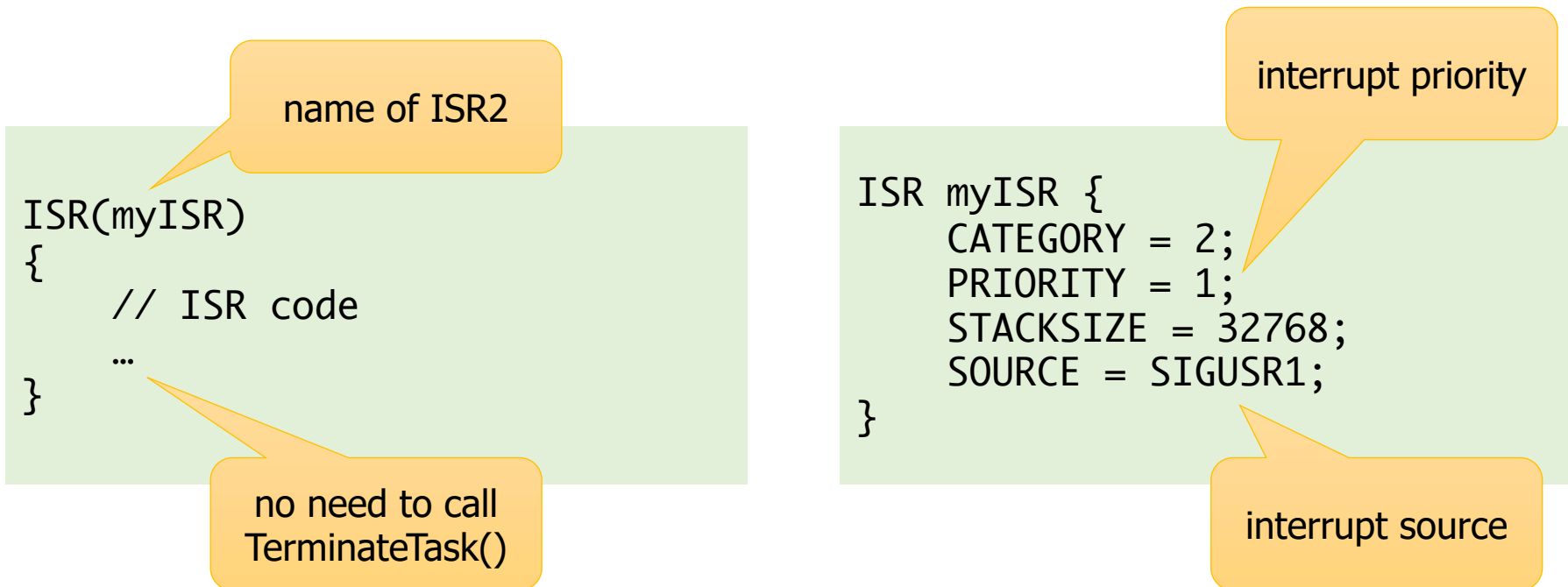


ISR1

- Low overhead / fast
- No system call allowed inside ISR1
- Ignored by the operating system
- No OIL description needed
 - Basically, ISR1 is not portable across different microprocessors

ISR2

- More overhead / slower than ISR1
- System calls allowed (ActivateTask, ...)
- Operating system is aware of ISR2
- OIL description needed



Questions

